August 18, 2026

1

Notebook zhijing-attempt-details- .csv

“ ” 0/100 Dice/F1 X_exact =100 1 0 Y CSV 1–5 “ ”

[37]:
```
from pathlib import Path
import json, math, warnings
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
from sklearn.decomposition import FactorAnalysis

warnings.filterwarnings('ignore', category=RuntimeWarning)
pd.set_option('display.max_columns', 100)
pd.set_option('display.max_rows', 200)
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'Arial Unicode␣
 ↪MS', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

#           CSV
CN = {
 'attempt_id':'  ','submitted_at':'  ','total_seconds':'    ','overall_score':
 ↪'    ','n_items':'  ','count':' ','unique':'  ','top':' ','freq':'  ','mean':
 ↪'  ','std':'  ','min':'  ','max':'  ',
 'mean_label':'    ','mean_strength':'    ','short_empirical':
 ↪'    ','low_empirical':'    ',
 'short_distribution':'    ','low_distribution':'    ','suspect':
 ↪'  ','review_reason':'  ','included_main':'   ',
 'item_id':'  ','position':'  ','n':'   ','n_administered':'  ','title':
 ↪'  ','modality':'  ','ability':'  ','option_type':'  ',
 'quota':'   ','pool_size':'   ','expected_n':'   ','actual_expected_ratio':
 ↪'   ','combination':'  ',
 'items':'  ','actual_min':'   ','actual_max':'   ','actual_mean':
 ↪'   ','within_cv':'    ',
```

```
 'exact_rate':'    ','exact_ci_low':'   95%  ','exact_ci_high':
 ↪'   95%  ','mean_partial_label':'     ',
 'mean_label_recall':'    ','mean_label_precision':'    ','mean_label_jaccard':
 ↪' Jaccard  ','mean_nonstandard_selection_rate':'      ',
 'corrected_item_total_r':'     ','D_27':' 27% D ','mean_seconds':
 ↪'     ','median_seconds':'      ','mean_item_score':'     ','mean_time':
 ↪'     ',
 'emotion':'  ','is_standard':'    ','selected_n':'  ','selector_rest_mean':
 ↪'      ','selection_rate':'  ',
 'mean_strength_match':'     ','sd_strength_match':
 ↪'     ','median_strength_match':'     ',
 'selected_emotion_ratings':'    ','mean_intensity':'    ','sd_intensity':
 ↪'    ','median_intensity':'     ',
 'q1':'    ','q3':'    ','floor_rate':'    ','ceiling_rate':
 ↪'    ','normalized_mean':'      ',
 'person_n':'    ','person_mean_intensity':'    ','person_sd_intensity':
 ↪'    ','standard_emotion_mean_intensity':
 ↪'    ','nonstandard_emotion_mean_intensity':'     ',
 'top_choice':'   ','top_choice_rate':'    ','correct_emotions':
 ↪'  ','top_is_standard':'       ',
 'initial_recommendation':'    ','expert_review_conclusion':
 ↪'    ','final_decision':'  ','review_notes':'  ',
 'n_all':'    ','exact_rate_all':'     ','partial_label_all':
 ↪'      ','strength_match_all':'     ',
 'exact_rate_difference':'    ','partial_label_difference':
 ↪'    ','strength_match_difference':'    '}
CN.update({f'level_{i}_n':f' {i}  ' for i in range(1,6)})
def chinese_table(frame):
    result=frame.rename(columns=CN,index=CN)
    if isinstance(result.index,pd.MultiIndex): result.index=result.index.
 ↪set_names([CN.get(n,n) for n in result.index.names])
    else: result=result.rename_axis(index=CN.get(result.index.name,result.index.
 ↪name))
    if isinstance(result.columns,pd.MultiIndex): result.columns=result.columns.
 ↪set_names([CN.get(n,n) for n in result.columns.names])
    else: result=result.rename_axis(columns=CN.get(result.columns.name,result.
 ↪columns.name))
    return result
def export_table(frame,path,index=False):
    chinese_table(frame).to_csv(path,index=index,encoding='utf-8-sig')

BASE_DIR = Path.cwd()
if not (BASE_DIR / 'zhijing-attempt-details- .csv').exists():
    BASE_DIR = BASE_DIR / 'predataAnalysis'
DATA_FILE = BASE_DIR / 'zhijing-attempt-details- .csv'
if not DATA_FILE.exists():
```

```
    raise FileNotFoundError(f'     {DATA_FILE.resolve()}')
OUTPUT_DIR = DATA_FILE.parent / '    '
OUTPUT_DIR.mkdir(exist_ok=True)
RANDOM_SEED = 20260818
print('   ', DATA_FILE.resolve())
print('   ', OUTPUT_DIR.resolve())
```

```
   /home/lyh/emotion-test/predataAnalysis/zhijing-attempt-details- .csv
   /home/lyh/emotion-test/predataAnalysis/
```

1.1 1.

NA

[38]:
```
raw = pd.read_csv(DATA_FILE, encoding='utf-8-sig')
rename = {
 '   ':'attempt_id','     ':'submitted_at','    % ':'attempt_score',
 ' ':'position','  ':'item_id','  ':'item_title','  ':'modality','  ':
 ↪'ability',
 '  ':'option_type','  ':'points','   ':'selected_emotions','   ':
 ↪'selected_strengths',
 '  ':'correct_emotions','   ':'standard_strengths','   ':'watched','  ':
 ↪'status',
 '   % ':'label_score','   % ':'strength_score','    % ':'item_score',
 '   ':'earned_points','    ':'response_seconds','  ':'modifications'}
missing_cols = sorted(set(rename) - set(raw.columns))
assert not missing_cols, f'CSV    {missing_cols}'
df = raw.rename(columns=rename).copy()
df['submitted_at'] = pd.to_datetime(df['submitted_at'], errors='coerce')
num_cols =␣
 ↪['attempt_score','position','points','label_score','strength_score','item_score','earned_po
df[num_cols] = df[num_cols].apply(pd.to_numeric, errors='coerce')
df['X_exact'] = np.isclose(df['label_score'], 100).astype(int)
df['label_partial_01'] = df['label_score'] / 100
df['strength_match_01'] = df['strength_score'] / 100
df['selected_list'] = df['selected_emotions'].fillna('').astype(str).str.
 ↪split(' ')
df['selected_strength_list'] = df['selected_strengths'].fillna('').astype(str).
 ↪str.split(' ')
df['correct_list'] = df['correct_emotions'].fillna('').astype(str).str.
 ↪split(' ')
df['standard_strength_list'] = df['standard_strengths'].fillna('').astype(str).
 ↪str.split(' ')

checks = pd.Series({
 '  ':len(df), '  ':df['attempt_id'].nunique(), '  ':df['item_id'].nunique(),
```

```
 '    ':df.groupby('attempt_id').size().min(), '    ':df.groupby('attempt_id').
 ↪size().max(),
 ' -   ':df.duplicated(['attempt_id','item_id']).sum(),
 '   ':(~df['status'].eq('  ')).sum(),
 '   ':(~df['label_score'].between(0,100)).sum(), '   ':(~df['strength_score'].
 ↪between(0,100)).sum(),
 '    ':sum(len(a)!=len(b) for a,b in␣
 ↪zip(df['selected_list'],df['selected_strength_list']))
})
display(checks.to_frame(' '))
assert checks[' -   '] == 0 and checks['    '] == checks['    '] == 24
```

```
         3264
          136
          147
         24
         24
-         0
          0
          0
          0
         0
```

1.2 2.

24 30 400 Q1 - 1.5×IQR
0

[39]:
```
attempts = (df.groupby('attempt_id').agg(submitted_at=('submitted_at','first'),␣
 ↪total_seconds=('response_seconds','sum'),
                overall_score=('attempt_score','first'),␣
 ↪n_items=('item_id','size'),
                mean_label=('label_score','mean'),␣
 ↪mean_strength=('strength_score','mean')).reset_index())
def lower_fence(s):
    q1, q3 = s.quantile([.25,.75]); return max(0, q1 - 1.5*(q3-q1))
time_cut = lower_fence(attempts['total_seconds'])
score_cut = lower_fence(attempts['overall_score'])
attempts['short_empirical'] = attempts['total_seconds'] < 400
attempts['low_empirical'] = attempts['overall_score'] < 30
attempts['short_distribution'] = attempts['total_seconds'] < time_cut
attempts['low_distribution'] = attempts['overall_score'] < score_cut
attempts['suspect'] = (attempts['short_empirical'] & attempts['low_empirical'])␣
 ↪| (attempts['short_distribution'] & attempts['low_distribution'])
```

```
attempts['review_reason'] = np.select([attempts['suspect'],␣
 ↪attempts['short_empirical']|attempts['short_distribution'],␣
 ↪attempts['low_empirical']|attempts['low_distribution']],
 ['     ','     ','     '], default='  ')
attempts['included_main'] = ~attempts['suspect']
print(f'    ={time_cut:.2f}      ={score_cut:.2f}   ={attempts.suspect.sum()}  ')
display(chinese_table(attempts.describe(include='all').T))
display(chinese_table(attempts.loc[attempts['review_reason']!='  '].
 ↪sort_values(['suspect','total_seconds'],ascending=[False,True])))
export_table(attempts,OUTPUT_DIR/'    .csv')
include_ids = set(attempts.loc[attempts['included_main'],'attempt_id'])
main = df[df['attempt_id'].isin(include_ids)].copy()
print('    ', main['attempt_id'].nunique(), '      ', df['attempt_id'].nunique())
```

```
   =0.00      =16.64   =8
```

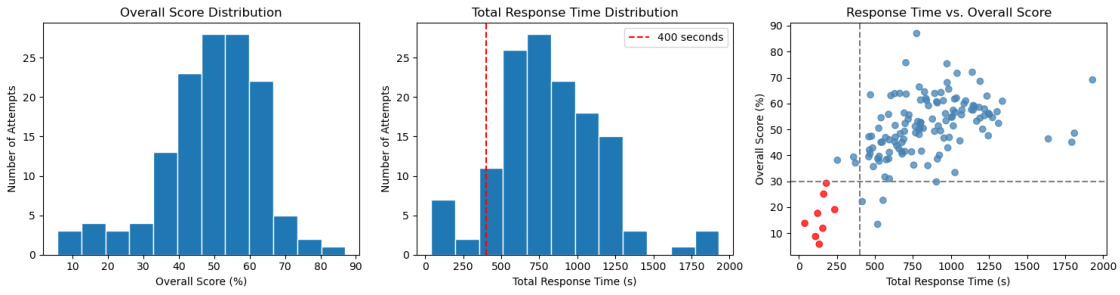
									\
	136	136	ZJ202608131494BDB9		1				NaN
	136	NaN			NaN	NaN	2026-08-16 04:43:08.823529472		
136.0	NaN				NaN	NaN		814.024412	
136.0	NaN				NaN	NaN		49.189191	
136.0	NaN				NaN	NaN		24.0	
136.0	NaN				NaN	NaN		52.583793	
136.0	NaN				NaN	NaN		43.671728	
	136	2		False	125			NaN	
	136	2		False	124			NaN	
	136	1		False	136			NaN	
	136	2		False	131			NaN	
	136	2		False	128			NaN	
	136	4			121			NaN	
	136	2		True	128			NaN	

		25%	50%	\
	NaN	NaN	NaN	
2026-08-13 20:16:00		2026-08-16 11:20:00	2026-08-16 11:35:00	
	38.84	593.6525	798.785	
	5.76	41.49	50.535	
	24.0	24.0	24.0	
	6.25	45.084479	54.27125	
	3.819583	35.503542	44.84375	
	NaN	NaN	NaN	
	NaN	NaN	NaN	
	NaN	NaN	NaN	
	NaN	NaN	NaN	
	NaN	NaN	NaN	
	NaN	NaN	NaN	
	NaN	NaN	NaN	

```
                          75%
                          NaN                    NaN         NaN
       2026-08-16 11:43:00  2026-08-16 13:24:00         NaN
                 1017.455              1930.76  336.391985
                  58.0575                 87.0   13.900229
                       24.0                 24.0         0.0
                 61.414271            91.666667   14.193313
                 52.907917                81.25    14.10437
                      NaN                  NaN         NaN
                      NaN                  NaN         NaN
                      NaN                  NaN         NaN
                      NaN                  NaN         NaN
                        NaN                  NaN         NaN
                        NaN                  NaN         NaN
                       NaN                  NaN         NaN

                                                          \
41   ZJ20260816136E1227 2026-08-16 11:20:00     38.84    13.80    24
20   ZJ2026081508C07905 2026-08-15 23:44:00    109.51     8.71    24
23   ZJ2026081530EFBB7A 2026-08-15 23:43:00    123.36    17.66    24
26   ZJ202608156B6462DB 2026-08-16 09:30:00    134.26     5.76    24
30   ZJ20260815B6992A82 2026-08-15 23:13:00    157.44    11.90    24
37   ZJ202608160A6834BD 2026-08-16 11:41:00    163.12    25.08    24
110  ZJ20260816C63CA8EE 2026-08-16 12:02:00    180.25    29.25    24
24   ZJ2026081555E6E754 2026-08-15 23:44:00    235.82    19.09    24
80   ZJ2026081676E36766 2026-08-16 11:45:00    253.48    38.13    24
29   ZJ20260815904AC83A 2026-08-16 00:08:00    359.85    39.37    24
133  ZJ20260816F9FDFC36 2026-08-16 10:45:00    372.01    37.08    24
84   ZJ202608168A889A87 2026-08-16 11:33:00    416.73    22.20    24
94   ZJ20260816A2D8D9D8 2026-08-16 11:28:00    517.88    13.43    24
100  ZJ20260816AE17A576 2026-08-16 12:29:00    553.66    22.68    24
6    ZJ202608135E4C7DD1 2026-08-13 23:31:00    904.13    29.78    24

                                                        \
41   17.916667   9.010417      True      True     False      True   True
20    9.722500   5.382083      True      True     False      True   True
23   19.722500  13.367917      True      True     False     False   True
26    6.250000   3.819583      True      True     False      True   True
30   14.166667  13.020833      True      True     False      True   True
37   28.055833  20.138750      True      True     False     False   True
110  35.695000  24.132083      True      True     False     False   True
24   20.833750  14.583333      True      True     False     False   True
80   40.139167  34.548750      True     False     False     False  False
29   42.520417  35.711667      True     False     False     False  False
133  44.047917  27.395833      True     False     False     False  False
84   25.000417  18.055417     False      True     False     False  False
94   17.361250  10.763750     False      True     False      True  False
100  25.139167  17.117917     False      True     False     False  False
```

```
6    32.460417  21.875000     False      True     False     False  False


41           False
20           False
23           False
26           False
30           False
37           False
110          False
24           False
80            True
29            True
133           True
84            True
94            True
100           True
6             True

     128        136
```

[40]:
```
fig, axes = plt.subplots(1,3,figsize=(15,4))
axes[0].hist(attempts['overall_score'], bins=12, edgecolor='white'); axes[0].
 ↪set(title='Overall Score Distribution',xlabel='Overall Score␣
 ↪(%)',ylabel='Number of Attempts')
axes[1].hist(attempts['total_seconds'], bins=12, edgecolor='white'); axes[1].
 ↪axvline(400,color='r',ls='--',label='400 seconds'); axes[1].set(title='Total␣
 ↪Response Time Distribution',xlabel='Total Response Time (s)',ylabel='Number␣
 ↪of Attempts'); axes[1].legend()
axes[2].scatter(attempts['total_seconds'],attempts['overall_score'],c=np.
 ↪where(attempts['suspect'],'red','steelblue'),alpha=.75); axes[2].
 ↪axvline(400,color='gray',ls='--'); axes[2].axhline(30,color='gray',ls='--');␣
 ↪axes[2].set(title='Response Time vs. Overall Score',xlabel='Total Response␣
 ↪Time (s)',ylabel='Overall Score (%)')
plt.tight_layout(); plt.show()
```



1.3 3.

‘ × / × / ’ 24 NaN

[41]:
```
#     ‘ × / × / ’      24/147
QUOTAS = {
 (' ',' ',' '):2, (' ',' ',' '):3, (' ',' ',' '):1,
 (' ',' ',' '):1, (' ',' ',' '):1, (' ',' ',' '):3, (' ',' ',' '):1,
 (' ',' ',' '):2, (' ',' ',' '):2, (' ',' ',' '):2,
 (' ',' ',' '):1, (' ',' ',' '):1, (' ',' ',' '):2, (' ',' ',' '):2}
expected_combo=pd.Series(QUOTAS,name='expected').
 ↪rename_axis(['modality','option_type','ability'])
observed_combo=(main.groupby(['attempt_id','modality','option_type','ability']).
 ↪size().unstack(['modality','option_type','ability'],fill_value=0).
 ↪reindex(columns=expected_combo.index,fill_value=0))
quota_violations=observed_combo.ne(expected_combo,axis=1).any(axis=1).sum()
print('          ',quota_violations)
assert quota_violations == 0, '          '
coverage = main.groupby('item_id').agg(n_administered=('attempt_id','nunique'),␣
 ↪title=('item_title','first'), modality=('modality','first'),␣
 ↪ability=('ability','first'), option_type=('option_type','first')).
 ↪reset_index()
coverage['quota']=[QUOTAS.get((m,o,a),0) for m,o,a in zip(coverage.
 ↪modality,coverage.option_type,coverage.ability)]
coverage['pool_size']=coverage.
 ↪groupby(['modality','option_type','ability'])['item_id'].transform('size')
coverage['expected_n']=main.attempt_id.nunique()*coverage.quota/coverage.
 ↪pool_size
coverage['actual_expected_ratio']=coverage.n_administered/coverage.expected_n.
 ↪replace(0,np.nan)
coverage['combination']=coverage.modality+' / '+coverage.option_type+' /␣
 ↪'+coverage.ability
cov_mean = coverage['n_administered'].mean(); cov_cv =␣
 ↪coverage['n_administered'].std(ddof=1)/cov_mean
combination_coverage=(coverage.
 ↪groupby(['combination','quota','pool_size','expected_n']).
 ↪agg(items=('item_id','size'),actual_min=('n_administered','min'),actual_max=('n_administere
 ↪x:x.std(ddof=1)/x.mean() if len(x)>1 else 0)).reset_index())
print(f'      ={coverage.n_administered.min()}  ={coverage.n_administered.
 ↪max()}   ={cov_mean:.2f}       ')
display(chinese_table(combination_coverage.round(3)))
position_table = pd.crosstab(main['position'],main['ability'])
display(chinese_table(position_table))
admin = (main.assign(v=1).
 ↪pivot_table(index='attempt_id',columns='item_id',values='v',aggfunc='max'))
co_n = admin.notna().astype(int).T.dot(admin.notna().astype(int))
tri = co_n.to_numpy()[np.triu_indices_from(co_n,1)]
```

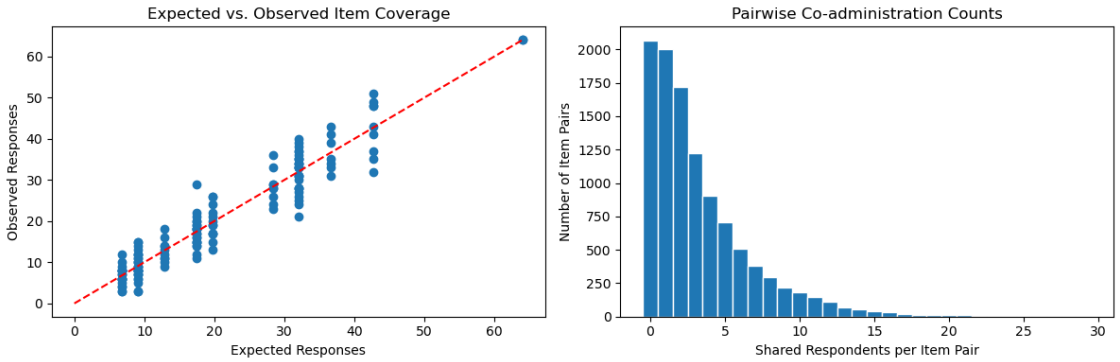
```
co_summary = pd.Series(tri).describe(percentiles=[.1,.25,.5,.75,.9,.95,.99]).
 ↪to_frame('    ')
display(co_summary); print('      ', f'{(tri==0).mean():.1%}', '  10     ',␣
 ↪f'{(tri>=10).mean():.1%}')
export_table(coverage,OUTPUT_DIR/'    .csv')
export_table(combination_coverage,OUTPUT_DIR/'     .csv')
export_table(pd.DataFrame(co_n),OUTPUT_DIR/'      .csv',index=True)
fig,axes=plt.subplots(1,2,figsize=(12,4)); axes[0].
 ↪scatter(coverage['expected_n'],coverage['n_administered']); lim=max(coverage.
 ↪expected_n.max(),coverage.n_administered.max()); axes[0].
 ↪plot([0,lim],[0,lim],color='r',ls='--'); axes[0].set(title='Expected vs.␣
 ↪Observed Item Coverage',xlabel='Expected Responses',ylabel='Observed␣
 ↪Responses'); axes[1].hist(tri,bins=np.arange(tri.max()+2)-.
 ↪5,edgecolor='white'); axes[1].set(title='Pairwise Co-administration␣
 ↪Counts',xlabel='Shared Respondents per Item Pair',ylabel='Number of Item␣
 ↪Pairs'); plt.tight_layout(); plt.show()
```

```
          0
     =3  =64   =20.90
```

										\
0	/	/		2	28	9.143	28	3	15	9.143
1	/	/		1	2	64.000	2	64	64	64.000
2	/	/		3	22	17.455	22	11	29	17.455
3	/	/		1	4	32.000	4	27	37	32.000
4	/	/		1	10	12.800	10	9	18	12.800
5	/	/		1	3	42.667	3	32	48	42.667
6	/	/		3	12	32.000	12	21	40	32.000
7	/	/		1	4	32.000	4	30	34	32.000
8	/	/		1	19	6.737	19	3	12	6.737
9	/	/		2	8	32.000	8	25	38	32.000
10	/	/		2	13	19.692	13	13	26	19.692
11	/	/		2	7	36.571	7	31	43	36.571
12	/	/		2	6	42.667	6	35	51	42.667
13	/	/		2	9	28.444	9	23	36	28.444

```
0       0.348
1       0.000
2       0.218
3       0.130
4       0.217
5       0.217
6       0.190
7       0.057
8       0.349
9       0.140
10      0.205
```

```
11      0.122
12      0.149
13      0.144


1       42    86
2       33    95
3       46    82
4       41    87
5       31    97
6       51    77
7       32    96
8       32    96
9       52    76
10      40    88
11      41    87
12      39    89
13      32    96
14      47    81
15      46    82
16      44    84
17      40    88
18      43    85
19      44    84
20      40    88
21      45    83
22      51    77
23      57    71
24      55    73


count  10731.000000
mean       3.292144
std        3.504027
min        0.000000
10%        0.000000
25%        1.000000
50%        2.000000
75%        5.000000
90%        8.000000
95%       10.000000
99%       16.000000
max       29.000000

      19.3%   10     6.6%
```



1.4 4.

X Wilson t

[42]:
```
def wilson(k,n,z=1.959964):
    if n==0: return (np.nan,np.nan)
    p=k/n; den=1+z*z/n; center=(p+z*z/(2*n))/den; half=z*np.sqrt(p*(1-p)/n+z*z/
 ↪(4*n*n))/den
    return center-half,center+half
def mean_ci(x):
    x=pd.Series(x).dropna().astype(float); n=len(x)
    if n<2:return (x.mean() if n else np.nan,)*2
    h=stats.t.ppf(.975,n-1)*x.std(ddof=1)/np.sqrt(n); return x.mean()-h,x.
 ↪mean()+h
display(chinese_table(attempts.
 ↪loc[attempts['included_main'],['overall_score','total_seconds','mean_label','mean_strength'
 ↪describe().T))
display(chinese_table(main.groupby(['ability','modality']).
 ↪agg(n=('attempt_id','size'),mean_item_score=('item_score','mean'),mean_time=('response_seco
 ↪round(2)))
```

```
                                                    25%          50%  \
    128.0    51.238125   11.398451   13.43000   43.560000   51.460000
    128.0   855.974375  299.923466  253.48000  632.745000  811.880000
    128.0    54.679945   11.568685   17.36125   47.187812   54.699583
    128.0    45.592969   12.065533   10.76375   36.657812   45.511875

                75%
     58.940000    87.000000
    1026.605000  1930.760000
     62.098125    91.666667
     53.276771    81.250000
```

```
128      42.74       26.60
  256      49.64       37.85
  384      48.07       47.65
  256      42.30       46.24
640      49.26       20.96
  512      57.23       28.36
  384      56.61       45.70
  512      52.29       40.73
```

1.5 5. X D

Dice/F1 Jaccard “ ” X_exact D 27% D

[43]:
```
#     rest score
ability_sum=main.groupby(['attempt_id','ability'])['X_exact'].transform('sum');␣
 ↪ability_n=main.groupby(['attempt_id','ability'])['X_exact'].transform('size')
main['X_rest_mean']=(ability_sum-main['X_exact'])/(ability_n-1)
def set_metrics(r):
    selected=set(x for x in r.selected_list if x); correct=set(x for x in r.
 ↪correct_list if x)
    hits=len(selected & correct); union=len(selected | correct)
    return pd.Series({'label_recall':hits/len(correct) if correct else np.
 ↪nan,'label_precision':hits/len(selected) if selected else 0,
      'label_jaccard':hits/union if union else np.
 ↪nan,'nonstandard_selection_rate':(len(selected)-hits)/len(selected) if␣
 ↪selected else 0})
main[['label_recall','label_precision','label_jaccard','nonstandard_selection_rate']]=main.
 ↪apply(set_metrics,axis=1)
def safe_corr(a,b,method='pearson'):
    z=pd.concat([pd.Series(a),pd.Series(b)],axis=1).dropna()
    if len(z)<5 or z.iloc[:,0].nunique()<2 or z.iloc[:,1].nunique()<2:return np.
 ↪nan
    return z.corr(method=method).iloc[0,1]
def item_x(g):
    n=len(g); k=int(g['X_exact'].sum()); lo,hi=wilson(k,n); q=max(1,int(np.
 ↪floor(.27*n))); s=g.sort_values('X_rest_mean'); low=s.head(q)['X_exact'].
 ↪mean(); high=s.tail(q)['X_exact'].mean()
    return pd.Series({'n':n,'exact_rate':k/n,'exact_ci_low':lo,'exact_ci_high':
 ↪hi,'mean_partial_label':g.label_partial_01.mean(),
      'mean_label_recall':g.label_recall.mean(),'mean_label_precision':g.
 ↪label_precision.mean(),'mean_label_jaccard':g.label_jaccard.
 ↪mean(),'mean_nonstandard_selection_rate':g.nonstandard_selection_rate.mean(),
      'corrected_item_total_r':safe_corr(g.X_exact,g.X_rest_mean),'D_27':
 ↪high-low,'mean_seconds':g.response_seconds.mean(),'median_seconds':g.
 ↪response_seconds.median()})
```

```
x_items=main.groupby('item_id',sort=False).apply(item_x).reset_index().
  ↪merge(coverage.drop(columns='n_administered'),on='item_id',how='left')
#    "      /    "          100%
choice_rows=[]
for item,g in main.groupby('item_id'):
    for _,r in g.iterrows():
        for emotion in r['selected_list']:
            if emotion: choice_rows.
  ↪append((item,emotion,r['X_rest_mean'],emotion in set(r['correct_list'])))
choices=pd.
  ↪DataFrame(choice_rows,columns=['item_id','emotion','rest','is_standard'])
option_analysis=(choices.groupby(['item_id','emotion','is_standard']).
  ↪agg(selected_n=('emotion','size'),selector_rest_mean=('rest','mean')).
  ↪reset_index().merge(coverage[['item_id','n_administered']],on='item_id'))
option_analysis['selection_rate']=option_analysis['selected_n']/
  ↪option_analysis['n_administered']
export_table(x_items,OUTPUT_DIR/' X  .csv');␣
  ↪export_table(option_analysis,OUTPUT_DIR/' X  .csv')
display(chinese_table(x_items.sort_values(['exact_rate','n']).head(20).
  ↪round(3)))
```

```
/tmp/ipykernel_3349/2412104346.py:19: FutureWarning: DataFrameGroupBy.apply
operated on the grouping columns. This behavior is deprecated, and in a future
version of pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or explicitly select
the grouping columns after groupby to silence this warning.
  x_items=main.groupby('item_id',sort=False).apply(item_x).reset_index().merge(c
overage.drop(columns='n_administered'),on='item_id',how='left')
```

				95%	95%	\	
80	ITEM-00029	6.0	0.000	0.000	0.390	0.000	
145	ITEM-00112	7.0	0.000	-0.000	0.354	0.000	
110	ITEM-00008	8.0	0.000	0.000	0.324	0.000	
33	ITEM-00046	12.0	0.000	0.000	0.242	0.553	
51	ITEM-00052	15.0	0.000	0.000	0.204	0.572	
8	ITEM-00043	17.0	0.000	-0.000	0.184	0.473	
97	ITEM-00126	17.0	0.000	-0.000	0.184	0.397	
126	ITEM-00066	17.0	0.000	-0.000	0.184	0.672	
11	ITEM-00065	20.0	0.000	0.000	0.161	0.535	
83	ITEM-00044	21.0	0.000	0.000	0.155	0.570	
2	ITEM-00076	24.0	0.000	0.000	0.138	0.370	
65	ITEM-00110	32.0	0.000	0.000	0.107	0.445	
85	ITEM-00080	35.0	0.000	0.000	0.099	0.455	
24	ITEM-00138	37.0	0.000	0.000	0.094	0.265	
22	ITEM-00139	64.0	0.031	0.009	0.107	0.428	
71	ITEM-00145	29.0	0.034	0.006	0.172	0.483	
75	ITEM-00098	28.0	0.036	0.006	0.177	0.544	
45	ITEM-00072	26.0	0.038	0.007	0.189	0.607	

43	ITEM-00107	25.0	0.040	0.007	0.195	0.508
54	ITEM-00069	24.0	0.042	0.007	0.202	0.630

				Jaccard		27% D \
80	0.000	0.000	0.000	1.000	NaN	0.000
145	0.000	0.000	0.000	1.000	NaN	0.000
110	0.000	0.000	0.000	1.000	NaN	0.000
33	0.625	0.575	0.422	0.425	NaN	0.000
51	0.556	0.691	0.447	0.309	NaN	0.000
8	0.368	0.765	0.319	0.235	NaN	0.000
97	0.441	0.420	0.278	0.580	NaN	0.000
126	0.603	0.865	0.547	0.135	NaN	0.000
11	0.438	0.797	0.386	0.203	NaN	0.000
83	0.548	0.683	0.425	0.317	NaN	0.000
2	0.396	0.395	0.265	0.605	NaN	0.000
65	0.531	0.414	0.320	0.586	NaN	0.000
85	0.364	0.681	0.316	0.319	NaN	0.000
24	0.284	0.268	0.181	0.732	NaN	0.000
22	0.406	0.516	0.319	0.484	0.100	0.059
71	0.500	0.531	0.361	0.469	0.164	0.143
75	0.500	0.723	0.418	0.277	-0.112	-0.143
45	0.500	0.874	0.468	0.126	-0.181	-0.143
43	0.480	0.620	0.387	0.380	0.370	0.167
54	0.569	0.799	0.492	0.201	-0.188	0.000

							\
80	16.355	15.170	34	2	28	9.143	
145	20.421	20.200	35	2	28	9.143	
110	24.995	23.185	30	2	28	9.143	
33	22.880	21.865	89	3	22	17.455	
51	21.077	15.920	94	3	22	17.455	
8	39.722	30.720	65	2	13	19.692	
97	29.975	27.870	86	2	13	19.692	
126	19.175	16.630	113	3	22	17.455	
11	21.435	19.060	112	3	22	17.455	
83	22.749	20.750	80	2	13	19.692	
2	32.679	28.975	78	2	9	28.444	
65	38.869	31.655	59	1	3	42.667	
85	45.214	37.880	65	2	8	32.000	
24	63.701	62.230	22	2	6	42.667	
22	26.255	22.040	37	1	2	64.000	
71	40.794	40.400	69	2	9	28.444	
75	38.288	32.955	1	2	8	32.000	
45	29.199	22.875	120	3	12	32.000	
43	40.958	37.510	81	2	8	32.000	
54	20.631	19.930	117	3	12	32.000	

```
80   0.656    /    /
145  0.766    /    /
110  0.875    /    /
33   0.688    /    /
51   0.859    /    /
8    0.863    /    /
97   0.863    /    /
126  0.974    /    /
11   1.146    /    /
83   1.066    /    /
2    0.844    /    /
65   0.750    /    /
85   1.094    /    /
24   0.867    /    /
22   1.000    /    /
71   1.020    /    /
75   0.875    /    /
45   0.812    /    /
43   0.781    /    /
54   0.750    /    /
```

1.6 6. Y

“ — ” normalized_intensity=(-1)/4 1 strength_score

[44]:
```
strength_rows=[]
for _,r in main.iterrows():
    for emotion,value in zip(r['selected_list'],r['selected_strength_list']):
        try: value=int(value)
        except: continue
        strength_rows.append({'attempt_id':r.attempt_id,'item_id':r.
 ↪item_id,'emotion':emotion,'intensity':value,'ability':r.
 ↪ability,'is_standard':emotion in set(r.correct_list)})
strength_long=pd.DataFrame(strength_rows);␣
 ↪strength_long['normalized_intensity']=(strength_long['intensity']-1)/4
level_counts=pd.
 ↪crosstab([strength_long['item_id'],strength_long['emotion']],strength_long['intensity']).
 ↪reindex(columns=range(1,6),fill_value=0).reset_index()
level_counts.columns=['item_id','emotion']+[f'level_{i}_n' for i in range(1,6)]
y_items=main.groupby('item_id').
 ↪agg(n=('attempt_id','size'),mean_strength_match=('strength_match_01','mean'),sd_strength_ma
 ↪reset_index()
```

```
raw_intensity=strength_long.groupby('item_id').
 ↪agg(selected_emotion_ratings=('intensity','size'),mean_intensity=('intensity','mean'),sd_in
 ↪x:x.quantile(.25)),q3=('intensity',lambda x:x.quantile(.
 ↪75)),floor_rate=('intensity',lambda x:(x==1).
 ↪mean()),ceiling_rate=('intensity',lambda x:(x==5).
 ↪mean()),normalized_mean=('normalized_intensity','mean')).reset_index()
person_intensity=(strength_long.groupby(['attempt_id','item_id']).intensity.
 ↪mean().rename('person_mean_intensity').reset_index().groupby('item_id').
 ↪agg(person_n=('attempt_id','size'),person_mean_intensity=('person_mean_intensity','mean'),p
 ↪reset_index())
standard_intensity=(strength_long.groupby(['item_id','is_standard']).intensity.
 ↪mean().unstack().rename(columns={True:
 ↪'standard_emotion_mean_intensity',False:
 ↪'nonstandard_emotion_mean_intensity'}).reset_index())
y_items=y_items.merge(raw_intensity,on='item_id',how='left').
 ↪merge(person_intensity,on='item_id',how='left').
 ↪merge(standard_intensity,on='item_id',how='left').merge(coverage.
 ↪drop(columns='n_administered'),on='item_id',how='left')
export_table(y_items,OUTPUT_DIR/' Y  .csv');␣
 ↪export_table(level_counts,OUTPUT_DIR/' Y    .csv')
display(chinese_table(y_items.sort_values('mean_strength_match').head(20).
 ↪round(3)))
```

						\	
7	ITEM-00008	8	0.000	0.000	0.00	24.995	8
28	ITEM-00029	6	0.000	0.000	0.00	16.355	6
108	ITEM-00112	7	0.000	0.000	0.00	20.421	7
116	ITEM-00120	8	0.094	0.265	0.00	25.562	8
34	ITEM-00036	7	0.107	0.283	0.00	28.186	7
110	ITEM-00114	9	0.111	0.333	0.00	18.172	9
14	ITEM-00015	6	0.125	0.306	0.00	47.055	6
20	ITEM-00021	6	0.125	0.306	0.00	54.960	6
6	ITEM-00007	13	0.135	0.333	0.00	17.256	13
39	ITEM-00041	18	0.139	0.323	0.00	18.414	18
109	ITEM-00113	7	0.143	0.378	0.00	22.407	7
58	ITEM-00060	11	0.182	0.337	0.00	30.105	11
139	ITEM-00143	37	0.182	0.298	0.00	36.184	37
43	ITEM-00045	31	0.194	0.321	0.00	46.249	31
113	ITEM-00117	10	0.200	0.422	0.00	24.399	10
134	ITEM-00138	37	0.209	0.245	0.25	63.701	69
8	ITEM-00009	35	0.214	0.285	0.00	29.169	35
115	ITEM-00119	8	0.219	0.411	0.00	21.312	8
27	ITEM-00028	3	0.250	0.433	0.00	22.627	3
24	ITEM-00025	4	0.250	0.289	0.25	36.995	4

					\		
7	3.250	0.707	3.0	3.00	4.00	0.000	0.000

```
28     3.833     1.169       4.0     3.25     4.75    0.000    0.333
108    4.000     0.577       4.0     4.00     4.00    0.000    0.143
116    4.125     0.641       4.0     4.00     4.25    0.000    0.250
34     3.143     1.069       3.0     3.00     4.00    0.143    0.000
110    3.889     0.601       4.0     4.00     4.00    0.000    0.111
14     4.667     0.516       5.0     4.25     5.00    0.000    0.667
20     3.833     0.753       4.0     3.25     4.00    0.000    0.167
6      3.769     0.832       4.0     3.00     4.00    0.000    0.154
39     3.778     0.943       4.0     3.00     4.00    0.000    0.222
109    4.143     0.378       4.0     4.00     4.00    0.000    0.143
58     3.909     0.831       4.0     3.00     4.50    0.000    0.273
139    4.297     0.878       4.0     4.00     5.00    0.027    0.486
43     4.161     0.820       4.0     4.00     5.00    0.000    0.387
113    4.300     0.483       4.0     4.00     4.75    0.000    0.300
134    3.667     0.934       4.0     3.00     4.00    0.014    0.159
8      3.743     0.741       4.0     3.00     4.00    0.000    0.114
115    3.250     0.463       3.0     3.00     3.25    0.000    0.000
27     4.667     0.577       5.0     4.50     5.00    0.000    0.667
24     4.500     1.000       5.0     4.50     5.00    0.000    0.750

                                                              \
7          0.562        8    3.250     0.707      3.250       NaN    30
28         0.708        6    3.833     1.169      3.833       NaN    34
108        0.750        7    4.000     0.577      4.000       NaN    35
116        0.781        8    4.125     0.641      4.143     4.000   105
34         0.536        7    3.143     1.069      3.167     3.000    45
110        0.722        9    3.889     0.601      3.875     4.000    38
14         0.917        6    4.667     0.516      4.600     5.000     4
20         0.708        6    3.833     0.753      3.600     5.000    14
6          0.692       13    3.769     0.832      3.818     3.500    29
39         0.694       18    3.778     0.943      3.867     3.333    54
109        0.786        7    4.143     0.378      4.167     4.000    36
58         0.727       11    3.909     0.831      3.750     4.333    18
139        0.824       37    4.297     0.878      4.192     4.545    71
43         0.790       31    4.161     0.820      4.091     4.333     2
113        0.825       10    4.300     0.483      4.375     4.000    47
134        0.667       37    3.730     0.729      3.583     3.857    22
8          0.686       35    3.743     0.741      3.667     3.857     7
115        0.562        8    3.250     0.463      3.167     3.500    49
27         0.917        3    4.667     0.577      4.500     5.000    24
24         0.875        4    4.500     1.000      4.000     5.000    60


7                  2     28   9.143  0.875    /    /
28                 2     28   9.143  0.656    /    /
108                2     28   9.143  0.766    /    /
116                2     28   9.143  0.875    /    /
34                 2     28   9.143  0.766    /    /
```

110	2	28	9.143	0.984	/	/
14	1	19	6.737	0.891	/	/
20	1	19	6.737	0.891	/	/
6	2	28	9.143	1.422	/	/
39	1	10	12.800	1.406	/	/
109	2	28	9.143	0.766	/	/
58	1	10	12.800	0.859	/	/
139	1	4	32.000	1.156	/	/
43	2	7	36.571	0.848	/	/
113	2	28	9.143	1.094	/	/
134	2	6	42.667	0.867	/	/
8	2	7	36.571	0.957	/	/
115	2	28	9.143	0.875	/	/
27	2	28	9.143	0.328	/	/
24	1	19	6.737	0.594	/	/

1.7 7.

[45]:
```
top_choice=(option_analysis.
 ↪sort_values(['item_id','selected_n'],ascending=[True,False]).
 ↪groupby('item_id').first().
 ↪reset_index()[['item_id','emotion','selection_rate']].
 ↪rename(columns={'emotion':'top_choice','selection_rate':'top_choice_rate'}))
diagnosis=x_items.merge(top_choice,on='item_id',how='left')
correct_map=main.groupby('item_id')['correct_emotions'].first();␣
 ↪diagnosis['correct_emotions']=diagnosis['item_id'].map(correct_map)
diagnosis['top_is_standard']=[str(c) in str(s).split(' ') for c,s in␣
 ↪zip(diagnosis.top_choice,diagnosis.correct_emotions)]
def recommend(r):
    if r.n < 15:return '      '
    if r.exact_rate < .25 and r.top_choice_rate >= .5 and not r.top_is_standard:
 ↪return '           '
    if r.exact_rate < .25 and pd.notna(r.corrected_item_total_r) and r.
 ↪corrected_item_total_r >= .2:return '        '
    if pd.notna(r.corrected_item_total_r) and r.corrected_item_total_r < 0:
 ↪return '            '
    if r.exact_rate < .25:return '          '
    if pd.notna(r.corrected_item_total_r) and r.corrected_item_total_r < .15:
 ↪return '        '
    return '   '
diagnosis['initial_recommendation']=diagnosis.apply(recommend,axis=1)
diagnosis['expert_review_conclusion']='  '; diagnosis['final_decision']='  ';␣
 ↪diagnosis['review_notes']='  '
export_table(diagnosis,OUTPUT_DIR/'      .csv')
```

```
display(chinese_table(diagnosis['initial_recommendation'].value_counts().
  ↪to_frame(' ')))
display(chinese_table(diagnosis.query('exact_rate < .25').
  ↪sort_values(['exact_rate','n']).head(30).round(3)))
```

```
                58
            29
              24
               17
                   10
                9
```

			95%	95%		
80	ITEM-00029	6.0	0.000	0.000	0.390	0.000
145	ITEM-00112	7.0	0.000	-0.000	0.354	0.000
110	ITEM-00008	8.0	0.000	0.000	0.324	0.000
33	ITEM-00046	12.0	0.000	0.000	0.242	0.553
51	ITEM-00052	15.0	0.000	0.000	0.204	0.572
8	ITEM-00043	17.0	0.000	-0.000	0.184	0.473
97	ITEM-00126	17.0	0.000	-0.000	0.184	0.397
126	ITEM-00066	17.0	0.000	-0.000	0.184	0.672
11	ITEM-00065	20.0	0.000	0.000	0.161	0.535
83	ITEM-00044	21.0	0.000	0.000	0.155	0.570
2	ITEM-00076	24.0	0.000	0.000	0.138	0.370
65	ITEM-00110	32.0	0.000	0.000	0.107	0.445
85	ITEM-00080	35.0	0.000	0.000	0.099	0.455
24	ITEM-00138	37.0	0.000	0.000	0.094	0.265
22	ITEM-00139	64.0	0.031	0.009	0.107	0.428
71	ITEM-00145	29.0	0.034	0.006	0.172	0.483
75	ITEM-00098	28.0	0.036	0.006	0.177	0.544
45	ITEM-00072	26.0	0.038	0.007	0.189	0.607
43	ITEM-00107	25.0	0.040	0.007	0.195	0.508
54	ITEM-00069	24.0	0.042	0.007	0.202	0.630
79	ITEM-00058	22.0	0.045	0.008	0.218	0.300
35	ITEM-00084	43.0	0.047	0.013	0.155	0.323
63	ITEM-00078	64.0	0.047	0.016	0.129	0.511
92	ITEM-00136	21.0	0.048	0.008	0.227	0.615
107	ITEM-00105	41.0	0.049	0.013	0.161	0.543
122	ITEM-00127	20.0	0.050	0.009	0.236	0.486
73	ITEM-00135	39.0	0.051	0.014	0.169	0.461
52	ITEM-00089	19.0	0.053	0.009	0.246	0.706
108	ITEM-00083	19.0	0.053	0.009	0.246	0.556
48	ITEM-00071	18.0	0.056	0.010	0.258	0.480

		Jaccard			27% D	
80	0.000	0.000	0.000	1.000	NaN	0.000

145	0.000	0.000	0.000	1.000	NaN	0.000
110	0.000	0.000	0.000	1.000	NaN	0.000
33	0.625	0.575	0.422	0.425	NaN	0.000
51	0.556	0.691	0.447	0.309	NaN	0.000
8	0.368	0.765	0.319	0.235	NaN	0.000
97	0.441	0.420	0.278	0.580	NaN	0.000
126	0.603	0.865	0.547	0.135	NaN	0.000
11	0.438	0.797	0.386	0.203	NaN	0.000
83	0.548	0.683	0.425	0.317	NaN	0.000
2	0.396	0.395	0.265	0.605	NaN	0.000
65	0.531	0.414	0.320	0.586	NaN	0.000
85	0.364	0.681	0.316	0.319	NaN	0.000
24	0.284	0.268	0.181	0.732	NaN	0.000
22	0.406	0.516	0.319	0.484	0.100	0.059
71	0.500	0.531	0.361	0.469	0.164	0.143
75	0.500	0.723	0.418	0.277	-0.112	-0.143
45	0.500	0.874	0.468	0.126	-0.181	-0.143
43	0.480	0.620	0.387	0.380	0.370	0.167
54	0.569	0.799	0.492	0.201	-0.188	0.000
79	0.341	0.284	0.225	0.716	0.185	0.200
35	0.360	0.332	0.243	0.668	0.544	0.182
63	0.539	0.554	0.383	0.446	-0.055	-0.059
92	0.587	0.687	0.491	0.313	-0.294	-0.200
107	0.504	0.689	0.415	0.311	0.618	0.182
122	0.450	0.575	0.352	0.425	-0.061	0.000
73	0.526	0.431	0.335	0.569	-0.012	0.000
52	0.719	0.775	0.572	0.225	-0.111	-0.200
108	0.579	0.596	0.413	0.404	0.660	0.200
48	0.528	0.463	0.351	0.537	0.195	0.250

\

80	16.355	15.170	34	2	28	9.143
145	20.421	20.200	35	2	28	9.143
110	24.995	23.185	30	2	28	9.143
33	22.880	21.865	89	3	22	17.455
51	21.077	15.920	94	3	22	17.455
8	39.722	30.720	65	2	13	19.692
97	29.975	27.870	86	2	13	19.692
126	19.175	16.630	113	3	22	17.455
11	21.435	19.060	112	3	22	17.455
83	22.749	20.750	80	2	13	19.692
2	32.679	28.975	78	2	9	28.444
65	38.869	31.655	59	1	3	42.667
85	45.214	37.880	65	2	8	32.000
24	63.701	62.230	22	2	6	42.667
22	26.255	22.040	37	1	2	64.000
71	40.794	40.400	69	2	9	28.444
75	38.288	32.955	1	2	8	32.000

```
45      29.199      22.875   120                3     12  32.000
43      40.958      37.510    81                2      8  32.000
54      20.631      19.930   117                3     12  32.000
79      23.650      19.575   102                3     22  17.455
35      32.731      25.690    78                2      6  42.667
63      26.938      22.360    89                1      2  64.000
92      25.944      24.060    71                3     12  32.000
107     46.129      42.890     2                2      6  42.667
122     69.155      60.075    87                2     13  19.692
73      31.838      28.120    68                3     12  32.000
52      62.474      58.660    84                2     13  19.692
108     82.266      57.630    73                2     13  19.692
48      30.099      30.855    91                3     22  17.455

                                                        \
80   0.656    /    /          0.333                     False
145  0.766    /    /           0.286                     False
110  0.875    /    /          0.125                     False
33   0.688    /    /           0.833                     True
51   0.859    /    /           0.667                    True
8    0.863    /    /           0.882                   True
97   0.863    /    /           0.765                     True
126  0.974    /    /           0.824                   True
11   1.146    /    /           0.750                   True
83   1.066    /    /           0.905                     True
2    0.844    /    /           0.667                     True
65   0.750    /    /           0.750                     True
85   1.094    /    /           0.771                   True
24   0.867    /    /           0.405                     True
22   1.000    /    /           0.641                     True
71   1.020    /    /           0.759                     True
75   0.875    /    /           0.750                    True
45   0.812    /    /           0.846                  True
43   0.781    /    /           0.760                     True
54   0.750    /    /            0.875                     True
79   1.260    /    /           0.364                     True
35   1.008    /    /           0.558                     True
63   1.000    /    /           0.797                     True
92   0.656    /    /           0.857                    True
107  0.961    /    /           0.659                    True
122  1.016    /    /           0.600                    True
73   1.219    /    /           0.564                     True
52   0.965    /    /           0.789                    True
108  0.965    /    /            0.842                      True
48   1.031    /    /           0.778                     True


80
```

145
110
33
51
8
97
126
11
83
2
65
85
24
22
71
75
45
43
54
79
35
63
92
107
122
73
52
108
48

1.8 8.

[46]:
```
sens=df.groupby('item_id').
  ↪agg(n_all=('attempt_id','size'),exact_rate_all=('X_exact','mean'),partial_label_all=('label
  ↪reset_index()
sensitivity=x_items[['item_id','n','exact_rate','mean_partial_label']].
  ↪merge(y_items[['item_id','mean_strength_match']],on='item_id').
  ↪merge(sens,on='item_id')
sensitivity['exact_rate_difference']=sensitivity['exact_rate']-sensitivity['exact_rate_all']
sensitivity['partial_label_difference']=sensitivity['mean_partial_label']-sensitivity['partial
sensitivity['strength_match_difference']=sensitivity['mean_strength_match']-sensitivity['stren
export_table(sensitivity,OUTPUT_DIR/'    .csv')
display(chinese_table(sensitivity.reindex(sensitivity.exact_rate_difference.
  ↪abs().sort_values(ascending=False).index).head(20).round(3)))
```

\

143	ITEM-00003	3.0	0.667	0.667	0.500	4	0.500
129	ITEM-00147	4.0	0.750	0.750	0.500	5	0.600
146	ITEM-00035	3.0	0.333	0.333	0.333	5	0.200
135	ITEM-00006	6.0	0.833	0.833	0.583	7	0.714
142	ITEM-00018	6.0	0.833	0.833	0.750	7	0.714
131	ITEM-00021	6.0	0.167	0.167	0.125	7	0.286
118	ITEM-00033	9.0	0.556	0.556	0.417	11	0.455
123	ITEM-00032	5.0	0.400	0.400	0.350	6	0.500
62	ITEM-00023	7.0	0.714	0.714	0.643	8	0.625
132	ITEM-00016	8.0	0.750	0.750	0.656	9	0.667
89	ITEM-00028	3.0	0.333	0.333	0.250	4	0.250
39	ITEM-00115	10.0	0.800	0.800	0.675	11	0.727
113	ITEM-00062	10.0	0.800	0.800	0.750	11	0.727
64	ITEM-00012	12.0	0.917	0.917	0.792	13	0.846
7	ITEM-00040	16.0	0.625	0.625	0.578	18	0.556
77	ITEM-00104	27.0	0.963	0.963	0.880	29	0.897
49	ITEM-00124	39.0	0.667	0.667	0.462	43	0.605
124	ITEM-00067	9.0	0.556	0.556	0.444	10	0.500
140	ITEM-00005	8.0	0.500	0.500	0.469	9	0.444
99	ITEM-00047	16.0	0.188	0.638	0.510	17	0.235

143	0.500	0.375	0.167	0.167	0.125
129	0.600	0.400	0.150	0.150	0.100
146	0.200	0.200	0.133	0.133	0.133
135	0.714	0.500	0.119	0.119	0.083
142	0.714	0.643	0.119	0.119	0.107
131	0.286	0.214	-0.119	-0.119	-0.089
118	0.455	0.341	0.101	0.101	0.076
123	0.500	0.417	-0.100	-0.100	-0.067
62	0.625	0.562	0.089	0.089	0.080
132	0.667	0.583	0.083	0.083	0.073
89	0.250	0.188	0.083	0.083	0.062
39	0.727	0.614	0.073	0.073	0.061
113	0.727	0.682	0.073	0.073	0.068
64	0.846	0.731	0.071	0.071	0.061
7	0.556	0.514	0.069	0.069	0.064
77	0.897	0.819	0.066	0.066	0.061
49	0.605	0.419	0.062	0.062	0.043
124	0.500	0.400	0.056	0.056	0.044
140	0.444	0.417	0.056	0.056	0.052
99	0.659	0.529	-0.048	-0.021	-0.019

1.9 9. EFA

EFA 24/147 80% 10 /IRT
CFA

[47]:
```
x_wide=main.pivot(index='attempt_id',columns='item_id',values='X_exact')
pair10=(tri>=10).mean(); connected=(co_n.where(~np.eye(len(co_n),dtype=bool),0).
 ↪gt(0).sum(axis=1)>0).all()
efa_ok = pair10 >= .80 and connected and x_wide.shape[0] >= 5*x_wide.shape[1]
efa_diag=pd.Series({'  ':x_wide.shape[0],'  ':x_wide.shape[1],'   ':x_wide.
 ↪shape[0]/x_wide.shape[1],'     ':np.median(tri),'    10  ':pair10,'     ':
 ↪connected,'  EFA ':efa_ok})
display(efa_diag.to_frame(' ')); efa_diag.rename_axis('  ').rename(' ').
 ↪to_csv(OUTPUT_DIR/'EFA   .csv',encoding='utf-8-sig')
if not efa_ok:
    print('       147      EFA        Notebook      ')
else:
    #                    /
    X=x_wide.apply(lambda c:c.fillna(c.median())).to_numpy()
    rng=np.random.default_rng(RANDOM_SEED); max_f=min(10,X.shape[1]-1); real=[];
 ↪ random=[]
    corr=np.corrcoef(X,rowvar=False); real=np.linalg.eigvalsh(corr)[::-1][:
 ↪max_f]
    for _ in range(100):
        xr=np.column_stack([rng.permutation(X[:,j]) for j in range(X.
 ↪shape[1])]); random.append(np.linalg.eigvalsh(np.corrcoef(xr,rowvar=False))[:
 ↪:-1][:max_f])
    random=np.mean(random,axis=0); n_f=max(1,int((real>random).sum()));␣
 ↪print('     ',n_f)
  ␣
 ↪fa=FactorAnalysis(n_components=n_f,rotation='promax',random_state=RANDOM_SEED).
 ↪fit(X)
    loadings=pd.DataFrame(fa.components_.T,index=x_wide.
 ↪columns,columns=[f'factor_{i+1}' for i in range(n_f)])
    loadings['primary_factor']=loadings.abs().idxmax(axis=1);␣
 ↪loadings['primary_loading']=loadings.lookup(loadings.
 ↪index,loadings['primary_factor']) if hasattr(loadings,'lookup') else␣
 ↪[loadings.loc[i,f] for i,f in loadings['primary_factor'].items()]
    loadings.to_csv(OUTPUT_DIR/'EFA  _     .csv',encoding='utf-8-sig');␣
 ↪display(chinese_table(loadings.head()))
```

```
                128
                147
          0.870748
             2.0
  10    0.066163
             True
EFA          False

    147      EFA        Notebook
```

1.10 10.

X/Y / EFA

[48]:
```
summary={
 '   ':int(df.attempt_id.nunique()),'   ':int(main.attempt_id.nunique()),' ':
 ↪int(len(df)),' ':int(df.item_id.nunique()),
 '  ':24,'   ':int(attempts.suspect.sum()),'    ':int(coverage.n_administered.
 ↪min()),
 '    ':int(coverage.n_administered.max()),'   ':
 ↪round(float(cov_mean),2),'  CV_ ':round(float(cov_cv),3),
 '    ':float(np.median(tri)),'    10 ':round(float(pair10),4),'EFA  ':
 ↪bool(efa_ok)}
(OUTPUT_DIR/'  .json').write_text(json.
 ↪dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
display(pd.Series(summary).to_frame(' '))
print('   '); [print(' -',p.name) for p in sorted(OUTPUT_DIR.iterdir())]
```

```
                136
               128
               3264
                147
                 24
                 8
                3
               64
              20.9
  CV_         0.619
               2.0
    10     0.0662
EFA             False


 - EFA   .csv
 -   .json
 -    .csv
 -    .csv
 -    .csv
 -  X  .csv
 -  X  .csv
 -  Y    .csv
 -  Y  .csv
 -     .csv
 -     .csv
 -   .csv
```

[48]: [None, None, None, None, None, None, None, None, None, None, None, None]